



PAK-AUSTRIA FACHHOCHSCHULE:
INSTITUTE OF APPLIED SCIENCES AND TECHNOLOGY

NAME: WAJID AYUB

REGISTRATION NUMBER: B23F0090AI136

SECTION: BS (AI RED)

SUBJECT: MACHINE LEARNING

PROGRAMME: ARTIFICIAL INTELLIGENCE

LAB COORDINATOR: SANA SALEEM

DATE: 27 Sept 2025

LAB NO: 11

Chapter 1

Introduction

The objective of this lab is to understand and implement the **ID3 Decision Tree Algorithm** from scratch. The ID3 algorithm is based on entropy reduction and information gain to recursively construct a decision tree.

According to the lab home task requirements, the following were completed:

- A new dataset containing marketing-related features was selected.
- The dataset was preprocessed into categorical attributes.
- A decision tree was implemented using the ID3 algorithm (without using sklearn).
- The decision tree was trained and tested on separate datasets.
- Model accuracy was measured for both training and test data.
- A visualization (text-based) of the decision tree was produced.

Dataset Used

Social Media Ads Dataset Link: <https://www.kaggle.com/datasets/rakeshrau/social-network-ads>

This dataset contains marketing-related user demographics and whether they purchased a product after seeing an ad.

Chapter 2

Dataset Preprocessing

2.1 Loading the Dataset

```
import pandas as pd

df =
pd.read_csv("Social_Network_Ads.csv")
print(df.head())
```

2.2 Selected Features

The dataset originally contains:

- Gender (Male/Female)
- Age
- Estimated Salary
- Purchased (Target)

ID3 requires all features to be **categorical**. Thus, continuous features were converted into bins:

- Age → Young / Middle / Senior
- Salary → Low / Medium / High

2.3 Encoding

```
df['AgeGroup'] = pd.cut(df['Age'],
                        bins=[18,30,40,60],
                        labels=['Young','Middle','Senior'])

df['SalaryGroup'] = pd.cut(df['EstimatedSalary'],
                           bins=[15000,50000,100000,150000],
                           labels=['Low','Medium','High'])

df['Gender'] = df['Gender'].astype('category')
df['Purchased'] = df['Purchased'].astype('category')
```

Unnecessary numeric columns were removed.

Chapter 3

ID3 Algorithm Implementation

3.1 Entropy

$$H(S) = - \sum_{i=1}^n p_i \log_2(p_i)$$

```
import numpy as np

def entropy(column):
    values, counts = np.unique(column,
    return_counts=True) probs = counts / counts.sum()
    return -np.sum(probs * np.log2(probs))
```

3.2 Information Gain

$$Gain(S, A) = H(S) - \sum_{v \in A} \frac{|S_v|}{|S|} H(S_v)$$

```
def information_gain(data, feature, target):
    total_entropy = entropy(data[target])
    values, counts = np.unique(data[feature], return_counts=True)

    weighted_entropy = 0
    for i, v in enumerate(values):
        subset = data[data[feature] == v]
        weighted_entropy += (counts[i]/len(data)) * entropy(subset[target])

    return total_entropy - weighted_entropy
```

3.3 ID3 Recursive Tree Builder

```
def id3(data, features, target):
    # If all samples belong to one class → leaf node
    if len(np.unique(data[target])) == 1:
        return np.unique(data[target])[0]

    # If no more features → return majority class
```

```

if len(features) == 0:
    return data[target].mode()[0]

# Compute gain for each feature
gains = {f: information_gain(data, f, target) for f in features}
best_feature = max(gains, key=gains.get)

tree = {best_feature: {}}
for value in np.unique(data[best_feature]):
    subset = data[data[best_feature] ==
                  value]

    if len(subset) == 0:
        tree[best_feature][value] = data[target].mode()[0]
    else:
        new_features = [f for f in features if f != best_feature]
        tree[best_feature][value] = id3(subset, new_features, target)

return tree

```

Chapter 4

Training and Testing

4.1 Train-Test Split

```
from sklearn.model_selection import train_test_split

train_df, test_df = train_test_split(df, test_size=0.3, random_state=0)

features = ['Gender', 'AgeGroup', 'SalaryGroup']
target = 'Purchased'

tree = id3(train_df, features, target)
print(tree)
```

Decision Tree (Text Visualization)

Gender

Male

AgeGroup = Young → 0

AgeGroup = Middle → 1

AgeGroup = Senior →

1

Female

AgeGroup = Young → 0

AgeGroup = Middle → 0

AgeGroup = Senior →

1

Chapter 5 Model Accuracy

A simple prediction function was written to traverse the decision tree.

```
def predict(tree, sample):
    feature = next(iter(tree))
    branch = sample[feature]
    result = tree[feature].get(branch,
0) if isinstance(result, dict):
        return predict(result, sample)
    else:
        return result
```

5.1 Training Accuracy

```
train_pred = train_df.apply(lambda row: predict(tree, row),
axis=1) train_acc = (train_pred == train_df[target]).mean()
print(train_acc)
```

5.2 Testing Accuracy

```
test_pred = test_df.apply(lambda row: predict(tree, row),
axis=1) test_acc = (test_pred == test_df[target]).mean()
print(test_acc)
```

Results

- **Training Accuracy: 0.8643**
- **Testing Accuracy:0.8583**

Chapter 6 Conclusion

This lab successfully demonstrated:

- Manual implementation of the ID3 decision tree algorithm
- Entropy and information gain calculations
- Recursive tree building without sklearn
- Tree-based decision rules learned from marketing data
- Accuracy evaluation on training and test datasets

The ID3 algorithm performed well on the categorical version of the Social Media Ads dataset, showing clear decision rules based on Gender, Age Group, and Salary Group.